
**School of Computer Science and Engineering**

**Faculty of Engineering**

**The University of New South Wales**

# Investigating the Impact of AI Tools on Teaching and Learning in Introductory Programming Courses

by

Matthew O'Dea

Thesis submitted as a requirement for the degree of

Bachelor of Engineering in Software Engineering

Submitted: August 2025

Supervisor: Prof. R Hamadi

Student ID: z5413887

# Abstract

The rapid adoption of generative Artificial Intelligence (AI) is reshaping introductory programming education, providing new opportunities to scaffold learning while simultaneously creating a crisis for academic integrity. Because AI tools can now generate functional code instantly from natural language prompts, they have made traditional assessments highly vulnerable to cheating. Furthermore, unguided use of these tools without pedagogical guardrails or structured instruction leads to a fundamental over-reliance, where students may outsource the problem-solving process entirely and bypass the critical cognitive struggles necessary for conceptual mastery.

This thesis focuses on building and testing two specific AI-driven interventions designed to strategically integrate these technologies into the curriculum. First, a course-grounded AI tutor was developed using Retrieval-Augmented Generation (RAG) to provide students with pedagogical guidance strictly aligned with official course materials, ensuring that the AI acts as a tutor rather than a solution-generator. Second, a scalable AI-driven oral assessment system was implemented to conduct interactive interviews, probing a student's conceptual understanding of their submitted work to ensure it reflects their own reasoning. These interventions were evaluated within the context of a pilot course at UNSW (COMP9021) using a mixed-methods approach. This evaluation combines quantitative interaction metrics, such as system usage patterns and performance data, with qualitative student surveys to assess the impact on learning outcomes, perceived fairness, and the effectiveness of the integration. By shifting the focus from the final code output to the authentic problem-solving process, this study seeks to enable the benefits of AI to scale while preserving the fundamental principles of computational thinking.

**Keywords:** Artificial Intelligence, Programming Education, Academic Integrity, Retrieval-Augmented Generation (RAG), Authentic Assessment, Mixed-Methods Evaluation

# Acknowledgements

I would like to express my sincere gratitude to my supervisor, Prof. Rachid Hamadi, for his expertise, consistent feedback, and guidance. His dedication and academic insight were essential to the development and progression of this thesis. I also thank Ali Darejah for his time and valuable feedback as an assessor.

This work is significantly informed by the researchers and scholars cited throughout this study, whose prior work established the foundation for this research. Additionally, I am grateful to my family and friends for their support. Finally, I would like to thank the UNSW School of Computer Science and Engineering (CSE) community for providing the environment and opportunities that made this education possible.

# Contents

# List of Figures

# List of Tables

# Chapter 1

# Introduction

Artificial Intelligence (AI) has advanced at a rapid rate in recent years, with generative models and tools such as ChatGPT, GitHub Copilot, and Amazon CodeWhisperer achieving rapid widespread adoption (Hu 2023). The pervasive integration of these technologies into higher education has occurred at a speed that necessitates a fundamental re-evaluation of foundational programming pedagogy (Alpizar-Chacon 2025). Although these tools offer the potential to scaffold understanding and increase learning efficiency, their unguided use threatens to undermine core educational goals, including conceptual mastery, problem-solving resilience, and academic integrity (Becker 2023).

This presents a critical challenge for educators: how to evolve pedagogy in a world where functional code can be generated instantly from natural language prompts. Simply banning these tools is both impractical and educationally regressive, as it denies students the opportunity to learn how to use the technologies that are reshaping the software development industry. In contrast, uncritical adoption risks producing "AI-dependent" students who outsource the entire problem-solving process to a Large Language Model (LLM), in the process bypassing the critical cognitive struggle required to achieve computational fluency (Bassner et al. 2026). Without pedagogical guardrails, students often lack the instruction necessary to use AI as a supportive tutor rather than a solution generator, leading to significant learning gaps that are often obscured until formal assessment.

Furthermore, the ability of AI to generate complete and correct code, particularly at an introductory level, has created an immediate crisis of academic integrity. Traditional assessment models, which often rely on the submission of code, are now highly vulnerable to automated cheating (Lau & Guo 2023). The central task for modern computer science education is therefore not to resist AI, but to strategically and responsibly integrate it into the curriculum through interventions that preserve the value of a degree.

This thesis confronts this challenge by building and testing two specific, scalable AI-driven interventions designed to strategically integrate these technologies into the introductory programming landscape:

- **A Course-Grounded AI Tutor:** Developed using Retrieval-Augmented Generation (RAG), this system provides students with pedagogical guidance strictly aligned with official course materials. This ensures that the AI acts as a mentor that supports learning processes rather than as a generator that provides direct answers.
- **An Automated Oral Assessment System:** This system leverages LLMs to conduct scalable, interactive "interviews." These sessions test a student's conceptual understanding of the work they submitted to ensure that the final product reflects their own reasoning and mastery.

The effectiveness of these interventions is evaluated through a mixed methods approach. This methodology combines quantitative interaction metrics, such as system usage patterns and performance data, with qualitative student surveys to assess the impact on learning, perceived fairness, and student trust.

The study uses *COMP9021: Principles of Programming* at the University of New South Wales (UNSW) as a pilot case study. As a postgraduate introductory course attracting a diverse cohort of students, many of whom are transitioning from non-computer science backgrounds, COMP9021 provides an ideal pilot environment. These students must rapidly acquire foundational skills, making them highly representative of the demographic most impacted by the shift toward AI-human collaboration in technology education.

## 1.1 Research Gaps and Questions

Two specific gaps in existing research motivate the interventions designed in this thesis.

**Lack of Deeply-Grounded AI Tutors:** While general AI assistants and tools such as CS50 Duck show promise, the existing literature evaluates tutors built on broad, general-purpose knowledge. There is a distinct gap in research on tutors grounded in a comprehensive, course-specific corpus — one that encompasses lecture slides, textbooks, past exam questions, coding style guides, and validated forum discussions from a single course. The pedagogical hypothesis is that deep, holistic grounding in a course's own authoritative materials enables qualitatively different guidance: explanations that use the lecturer's precise terminology, practice problems modelled on the actual exam style, and answers that cannot be obtained from a general-purpose AI.

**Absence of a Scalable, Pedagogical Oral Assessment Model:** Traditional assessments are increasingly vulnerable to AI misuse, yet oral examinations — the most robust alternative — are not scalable in large courses. Industry interview platforms address the scheduling problem but are not designed for a pedagogical context: they do not assess a student's understanding of their own previously submitted work, nor do they probe for conceptual knowledge aligned with a course curriculum. What is needed is a system that conducts structured, one-on-one dialogues in which students explain their own submitted code and answer related conceptual questions — shifting the assessment focus from "what did you produce?" to "how do you think?"

These gaps lead to the following central problem: how can AI tools be strategically and responsibly integrated into introductory programming courses to enhance learning outcomes, preserve core educational values, and align with emerging industry norms? This thesis addresses this problem through three research questions:

1. To what extent does a course-grounded AI tutor built on course-specific materials improve students' conceptual understanding and learning experience compared to general-purpose AI tools?

2. Can a scalable AI-driven oral assessment system effectively evaluate a student's program-

ming competency by probing their reasoning and understanding of their own submitted code, and does it strengthen academic integrity compared to traditional assessments?

3. How do students perceive the usefulness, fairness, and trustworthiness of AI-based tutoring and assessment systems in a foundational programming course?

To answer these questions, this study designed, deployed, and evaluated two distinct AI interventions within UNSW's COMP9021, providing an empirical basis for a new model of AI-augmented programming pedagogy.

# Chapter 2

# Background

## 2.1 The Paradoxical Nature of AI in Programming Education

The integration of AI into programming education presents a fundamental challenge. Although AI tools offer significant pedagogical benefits — by automating routine tasks such as syntax correction or boilerplate code generation, they can reduce extraneous cognitive load, allowing students to focus on core programming concepts (Bettayeb et al. 2024). AI tutors can also act as adaptive learning aids, offering personalised explanations that can be particularly valuable in situations where personalised instruction is inaccessible, such as large-scale online or distance learning environments (Edwards-Fapohunda & Adediji 2024, Zviel-Girshin 2024).

Conversely, these same capabilities introduce profound pedagogical risks. The primary concern is the potential for students to develop an uncritical over-reliance on AI, leading to a superficial understanding of programming logic (Elsayed 2024). Students may use LLMs to generate solutions without cognitively engaging in the essential process of problem-solving, debugging, and critical thinking. Furthermore, the outputs of large language models are not infallible; they can produce code that is subtly incorrect or explanations that are misleading, yet deliver them with an authoritative tone that a novice learner is ill-equipped to question (Zviel-Girshin 2024). This dual nature of AI necessitates a pedagogical approach that actively cultivates critical evaluation skills alongside tool proficiency.

Programming education rests on a set of foundational principles that no pedagogical approach — however technologically mediated — can safely discard. The ACM CS2023 curriculum guidelines identify the core competencies students must develop: algorithmic thinking, data structure design, syntactic fluency, and the capacity to trace logical execution flow (Raj et al. 2023). These skills require deliberate cognitive engagement; they cannot be passively acquired through AI-generated solutions. Beyond foundations, programming education must also cultivate higher-order capabilities — critical analysis, systematic problem-solving, and creative design — which represent precisely the cognitive work that AI tools are most capable of bypassing (Cambaz & Zhang 2024).

## 2.2 The Crisis of Assessment and Academic Integrity

The capabilities of modern AI have rendered many traditional assessment methods in programming education obsolete. Analysis of ChatGPT's performance on standard CS curriculum assessments found it capable of handling a wide range of introductory programming tasks (Qureshi 2023), and subsequent generations of large language models have only widened this gap (Huckins 2025). The result is that code submission — the dominant assessment mode in introductory courses — now provides little assurance of authentic student work, directly threatening the credibility of academic credentials (Humble et al. 2024).

Efforts to combat this, such as AI-detection software, have proven unreliable and impractical, especially for the small functional code segments typical of introductory courses (Finnie-Ansley et al. 2022). Enforcement-based strategies that ban AI usage are not only difficult to police but also create an adversarial dynamic: students who perceive peers gaining an advantage through AI use face growing pressure to do the same, eroding academic norms from within. This situation has created an urgent need to fundamentally rethink assessment design, moving away from tasks that can be easily outsourced to an AI and toward methods that evaluate a student's authentic problem-solving process.

## 2.3 Emerging Pedagogical Responses and Their Limitations

In response to this crisis, two divergent philosophies are emerging. A protectionist approach advocates for designing "AI-proof" assessments — handwritten code, oral examinations, or complex multimodal projects that are difficult for current AI to solve independently (Lau & Guo 2023). While this strategy can preserve the integrity of certain assessments, it is not a holistic solution. Such tasks risk being detached from the collaborative, tool-augmented contexts in which students will actually write code professionally. They also represent a temporary fix: as large language models continue to advance, tasks considered AI-resistant today will not remain so indefinitely. Worse still, an adversarial framing incentivises students to become more sophisticated in evading detection rather than more capable as programmers.

An integrationist approach argues that AI should be embedded in both course content and delivery, treating these tools as objects of study rather than threats to be contained (Cambaz & Zhang 2024).

The most prominent example is Harvard's CS50 course, which deployed the CS50 Duck, an AI tutor built on GPT-4 (Liu et al. 2024). As seen in Figure 2.1, a custom OpenAI GPT-4 wrapper with CS50-specific contextual knowledge provided via retrieval-augmented generation (RAG) endowed students with a real-time context-aware tutor. The tutor is designed to guide students with hints rather than providing direct answers. Adoption and satisfaction rates were exceptionally high, with over 80

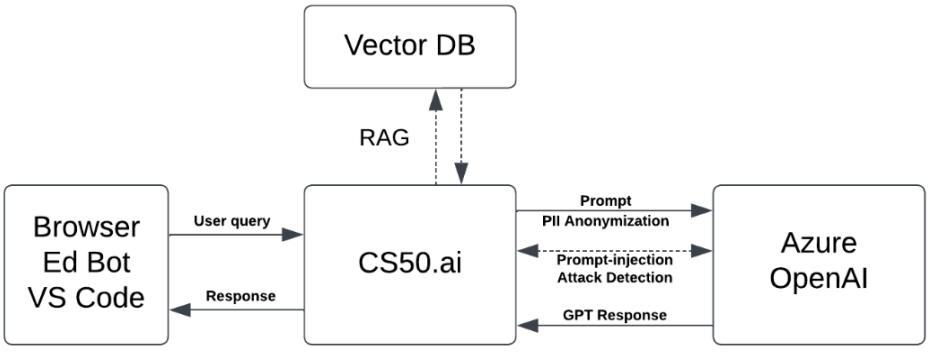


Figure 2.1: High-level overview of the system architecture used in CS50 Duck

However, the CS50 model, while successful in its context, has key limitations. There is a need to move beyond general AI assistants and investigate the efficacy of tutors that are deeply integrated with a specific course's entire ecosystem of materials, from lecture slides and textbooks to coding style guides and the history of faculty-approved answers on course forums. Such a tool could explain a difficult concept using the precise terminology of the lecturer or generate new practice problems modelled on the style of past exams, offering a truly cohesive learning experience.

## 2.4 Developing Scalable, Authentic Assessments

The main obstacle impeding widespread adoption of oral assessments is scalability. Conducting individual interviews is prohibitively time-consuming for instructors in large courses.

Automated feedback on code structure and functionality has proven both possible and successful, with various models capable of evaluating code quality, structure, and similarity to test cases (Gambo et al. 2024, Parvathy et al. 2025). A representative model includes three main components, as shown in Figure 2.2:

- **Syntax Error Detection Module (SEDM)**: Detects common syntax errors in code.
- **Structure-Based Code Assessment Model (SBCAM)**: Assesses code organisation, readability, and maintainability.
- **Similarity and Test-Case Based Code Assessment Model (STCAM)**: Evaluates functionality through test cases and checks for code similarity to uphold academic integrity.

These systems demonstrate that code can be evaluated automatically, allowing for tailored follow-up questions to be LLM-generated to test a student's understanding. However, these models primarily assess the final product rather than the problem-solving process.

More recently, intelligent agents have been developed to automate industry-style technical interviews, using metrics such as BLEU and BERTScore to evaluate candidate responses (Joseph & Judy 2024). These tools are already widespread in industry, with platforms like HireVue

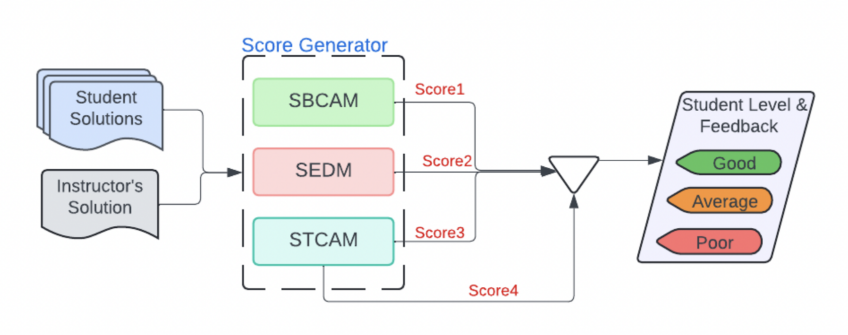


Figure 2.2: Overview of the automated code assessment system components

conducting entry-level role interviews across various sectors and evaluating candidates' technical understanding. However, these industry-focused agents are not designed for a pedagogical context. They lack the framework to assess a student's understanding of their own previously completed work or to probe for conceptual knowledge related to a course curriculum.

## 2.5 AI Solutions for Education

The integrationist approaches described above — pedagogically constrained AI tutors and automated oral assessment — both depend on AI architectures that enable course-specific, reliable, and educationally bounded behaviour. Two foundational technologies underpin the system designs explored in this thesis: Retrieval-Augmented Generation (RAG), which grounds AI responses in a curated corpus of course materials, and the Model Context Protocol (MCP), which provides a standardised framework for delivering structured context to AI components.

### 2.5.1 Retrieval-Augmented Generation (RAG)

RAG combines a large language model (LLM) with an external knowledge source to improve accuracy, reduce hallucinations, and enable domain-specific responses (Lewis et al. 2021). In this approach, relevant documents are retrieved from a structured knowledge base or vector store and provided to the model as additional context during generation. This allows the system to incorporate information beyond the model's static training data, delivering more accurate

and contextually relevant outputs (Min et al. 2022). RAG has a wide range of applications. It can be used to perform real-time searches over institution documentation, policies, or technical repositories, enabling instant access to relevant information. Figure 2.3 displays a potential implementation of RAG.

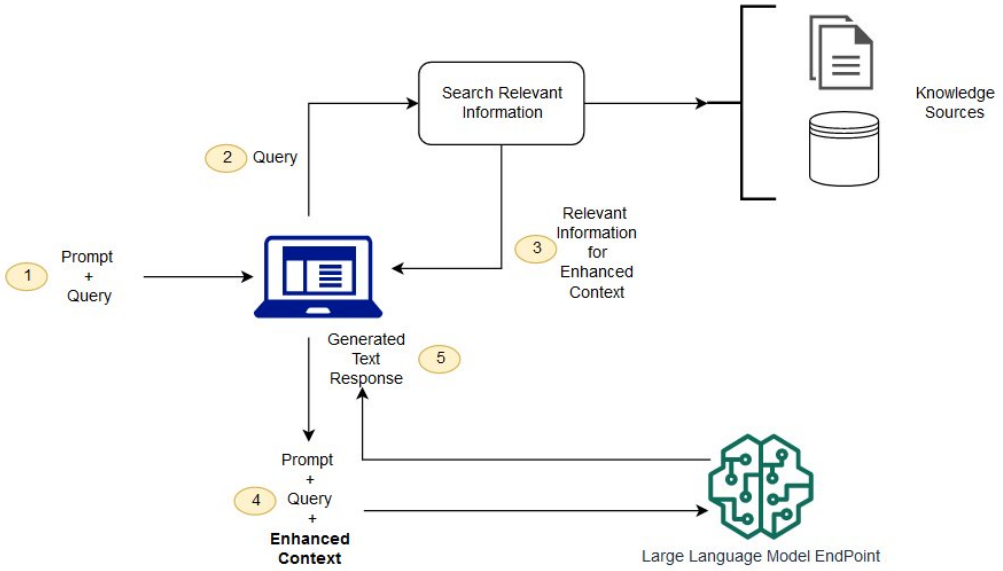


Figure 2.3: High-level architecture of Retrieval-Augmented Generation (RAG)

However, retrieval adds extra processing steps, which can increase latency and overall system complexity. In addition, the indexing pipelines, embeddings, and vector databases that underpin RAG must be maintained continuously. As a result, without regular maintenance, the quality and relevance of the retrieved information can degrade over time.

### 2.5.2 Model Context Protocol (MCP) Servers

MCP servers provide a standardised framework for managing and delivering context to large language models (LLMs). They act as middleware between the AI model and its information sources, packaging relevant course content, user data, and system instructions into a structured format (Hou et al. 2025). This enables consistent, context-aware interactions across different AI components, and ensures that all systems are grounded in the same authoritative data. MCP can facilitate multi-step workflows by chaining together different tools, enabling secure

collaboration between systems and agents by allowing them to access and share information efficiently (Ahmadi et al. 2025). A sample implementation of an MCP server is shown in figure 2.4.

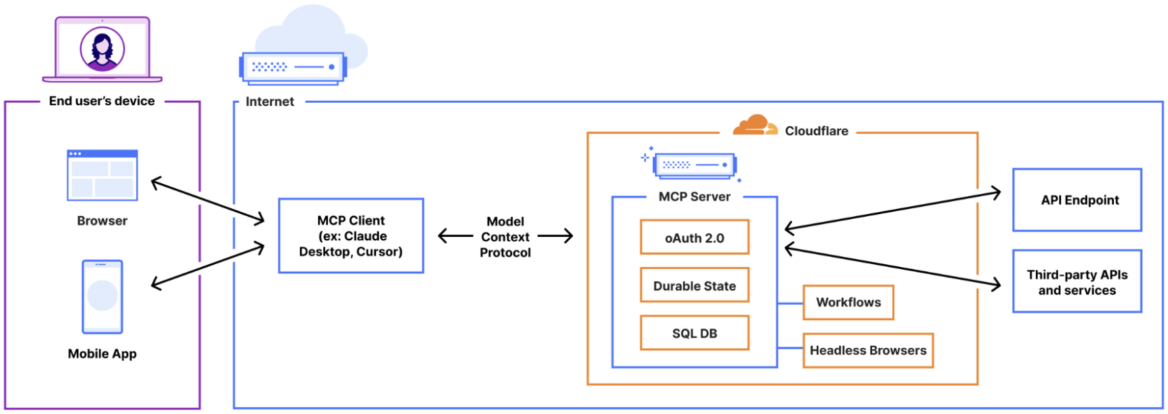


Figure 2.4: Example architecture for an authenticated MCP server connecting multiple AI components with shared context.

However, the MCP integration process is complex, which may result in increased operational costs depending on deployment scale and infrastructure choices. Additionally, the performance of the tool degrades rapidly as the number of external integrations increases.

# Chapter 3

# System Design and Implementation

This chapter describes the design and implementation of the two systems built for this study: a course-grounded AI tutor and an automated oral assessment platform. Both share a common FastAPI backend deployed on AWS, with separate React frontends serving distinct user roles.

## 3.1 System Architecture

The platform follows a strict three-tier architecture. A single FastAPI backend (Python 3.11) exposes all functionality through a RESTful API, with layered separation between HTTP routing (`controllers/`), business logic (`service/`), and data contracts (`dtos/`). Three React frontends communicate exclusively with this backend:

- **AI Tutor Frontend:** A general-purpose learning interface offering RAG-powered chat and an in-browser Python code editor.
- **Instructor Dashboard:** A management interface for creating assessments, generating questions, and reviewing student results.
- **Student Assessment Interface:** A guided interface for completing oral assessments, recording audio responses, and viewing feedback.

Core infrastructure runs on AWS: Amazon Bedrock supplies language model inference, DynamoDB provides persistent storage, S3 stores audio recordings, and Neo4j serves as the vector database for retrieval. All resources are provisioned via Terraform, and deployment is automated through a GitHub Actions CI/CD pipeline.

## 3.2 The AI Tutor

### 3.2.1 Retrieval-Augmented Generation Pipeline

The AI tutor grounds all responses in a corpus of COMP9021 course materials using Retrieval-Augmented Generation (RAG). Rather than relying on the general knowledge of an LLM, the system retrieves relevant course-specific content at query time and injects it into the model's context, ensuring that responses reflect the course's terminology, approved content, and pedagogical approach.

#### Knowledge Base Construction

Course materials — lecture slides, lab specifications, assignment briefs, and coding style guides — are ingested through a document processing pipeline. Each document is chunked into semantically coherent segments and embedded into a 1024-dimensional vector space using the Cohere Embed English v3 model via Amazon Bedrock. These embeddings, along with structured metadata, are stored as nodes in a Neo4j graph database.

The chunking strategy is content-aware. Markdown documents are split at heading boundaries to preserve logical sections; Python source files are split at function and class definitions; documents without structural markers fall back to size-based chunking with a maximum of 1,800 characters per chunk and 220 characters of overlap to preserve context across boundaries. Each chunk stores its embedding vector, a scope tag (e.g. `comp9021_T1_2026`) for query filtering, and provenance metadata including artifact type, content family, module number, and source path.

**Semantic Search**

At query time, the user's message is embedded with the same Cohere model and compared against all stored chunks using cosine similarity. The top-five chunks by similarity score are selected and assembled into a context block injected into the model's prompt, capped at 8,000 characters to remain within the model's input window.

**Chat Pipeline**

The full pipeline from user message to response proceeds as follows:

1. The user message is received by the `ChatService` with session state.
2. Up to ten prior messages from the session history are retrieved and truncated to 500 characters each.
3. The message is embedded and the top-five relevant document chunks are retrieved from Neo4j.
4. A system prompt is constructed from the base tutor instructions, the active pedagogy mode, and the retrieved context.
5. The assembled prompt is sent to Amazon Nova Lite (model ID: `amazon.nova-lite-v1:0`) via the Bedrock Runtime API.
6. The response is returned to the client with token usage, context chunk IDs, and session metadata.

### 3.2.2 Pedagogical Design

The tutor exposes two interaction modes, selectable by the student.

**General Chat** (*explanatory mode*) targets conceptual understanding. The system prompt instructs the model to provide structured explanations with examples, cap responses at approximately 250 words, and highlight common misconceptions. For assessed work, the model

is explicitly prohibited from generating complete solutions; instead, it is instructed to scaffold understanding and guide the student toward the answer.

**Code Assistant** (*concise mode*) targets implementation support. Responses are shorter and more direct, formatted as bullet points or short paragraphs. The same academic integrity constraints apply: for assessed tasks, the model provides hints and partial scaffolding rather than working code.

Both modes share a hard constraint enforced at the prompt level: the model must not claim that any output is ready to submit, and must refuse requests that amount to outsourcing the problem.

### 3.2.3 Student Interface

The AI tutor frontend provides two views. In chat mode, students interact through a message interface with persistent session history. In IDE mode, a Monaco-based code editor is integrated alongside the chat panel, supporting Python syntax highlighting, selection tracking, and in-browser execution via Pyodide, allowing students to run code without a server round-trip. When a student submits a message in IDE mode, the current code, any selected region, and the most recent execution output are included as context alongside the query, enabling targeted, code-specific responses.

## 3.3 The Automated Oral Assessment System

### 3.3.1 Question Generation

For each assessment, an instructor uploads a student's code submission and the relevant assignment brief. The `QuestionGenerationService` submits these to Amazon Nova Lite with a structured prompt specifying an exact output schema: eight questions per student comprising five specific questions about the student's own code and three general conceptual questions aligned with the assignment's learning objectives.

The prompt enforces quality constraints — questions must be answerable in one to two minutes, must not be yes/no, and must address one concept each. Output is returned as a strict JSON array and stored in DynamoDB, keyed by student and assessment ID. An overview of the full assessment system architecture is shown in Figure 4.2.

### 3.3.2 Student Assessment Interface

The student frontend guides students through a sequential question flow. For each question, the student reads the question text and records an audio response using the browser's MediaRecorder API. On submission, the audio file is uploaded directly to S3 using a presigned URL issued by the backend, avoiding routing binary data through the API server. The S3 URL and recording duration are then submitted to the backend and persisted in DynamoDB.

### 3.3.3 Speech-to-Text Transcription

Audio responses are transcribed using the Deepgram REST API with the general-purpose English model. The backend retrieves the audio from S3 and submits it as a binary stream to Deepgram's `/v1/listen` endpoint with a 120-second timeout. An optional post-processing step uses Amazon Nova Lite to correct punctuation and casing in the raw transcript before it is passed to the evaluation engine.

### 3.3.4 Evaluation Engine

Student responses are evaluated by a structured LLM-based rubric implemented in the `ResponseEvaluationEng`
Each question-answer pair is submitted to Amazon Nova Lite with a prompt defining a two-dimensional scoring framework:

- **Correctness** (0–5): assesses the factual and technical accuracy of the response.
- **Understanding** (0–5): assesses depth of conceptual explanation, rewarding responses that articulate *why* or *how* rather than merely *what*.

The prompt explicitly instructs the model to assess content quality rather than speaking fluency, penalising disfluency only where it obscures meaning. The model returns a strict JSON object containing both scores, a list of up to three strengths and weaknesses, two to three sentences of feedback, and one to three suggested improvements. The total score out of ten maps to one of four grade labels: *Excellent* (80%+), *Competent* (60–79%), *Developing* (40–59%), and *Beginning* (below 40%).

Evaluation is processed asynchronously via a DynamoDB-backed job queue (`BatchJobManager`), allowing multiple students' assessments to be evaluated concurrently without blocking the request cycle.

### 3.3.5 Instructor Dashboard

The instructor frontend provides assessment creation, student enrolment, progress monitoring, and results review. Once evaluation jobs complete, per-student results are aggregated from DynamoDB and presented as a table of scores, grades, and per-question feedback. Individual results are also available as generated PDF exports.

## 3.4 Infrastructure and Deployment

The FastAPI application runs in a Docker container on an EC2 instance managed by Docker Compose, with a systemd service ensuring automatic restart on failure. A four-stage GitHub Actions CI/CD pipeline enforces quality on every commit: backend tests using pytest with moto-mocked AWS services, frontend type-checking and linting, Vitest unit tests, and a Playwright end-to-end suite running against a mock API. Deployment is gated on all four stages passing.

JWT-based authentication protects all API endpoints, with access controls ensuring students can only access their own assessment data.

# Chapter 4

# Methodology

## 4.1 Project-Dependent Skills

Having established the research problem, this chapter details the groundwork required for the technical and ethical implementation of the proposed systems. This section will detail the techniques and knowledge necessary for a successful implementation of the project.

Overview - build, trial and evaluate systems within pilot course at unsw. From feb - april 2026 (unsw trimester 1). full rollout of AI tutor. limited voluntary trial of oral assessments to 1. check efficacy and 2. gather feedback to address and trial ethical, privacy and fairness concerns

## 4.2 Software Development Methodology

The project will adopt an agile-inspired development methodology, emphasising iterative prototyping and continuous evaluation. This research will form the basis for further experimentation and development. Agile methods are particularly well-suited due to their adaptability to evolving requirements, user feedback, and emerging technological capabilities (Beck 2001, Lucia & Qusef 2010).

Recognising that this is a living project impacting students' education, quality and continuity are paramount. The following practices were prioritised:

- **Solid and Thorough Testing:** Unit tests, integration tests, and regression tests will be incorporated into the CI/CD pipeline to ensure functional correctness and prevent the reintroduction of bugs.
- **Performance Monitoring and Logging:** Application metrics and AI inference performance will be monitored in real time, with logs stored securely for audit and troubleshooting purposes.
- **Documentation and Knowledge Management:** Technical documentation will be maintained in parallel with development, covering architecture diagrams, API specifications, and operational guides to ensure maintainability. - anything else relevant

## 4.3 Project Ethical Considerations

The project will follow UNSW's *Human Research Ethics Guidelines* (UNSW 2025) and the *Australian National Statement on Ethical Conduct in Human Research*, ensuring AI tools in education are deployed fairly, transparently, and securely.

**Data Privacy and Security** — All student data will be stored securely in compliance with the Australian Privacy Principles under the *Privacy Act 1988*. Access will be restricted to authorised personnel, and retention will follow UNSW's records management policy.

**Informed Consent** — Participation in trials will be voluntary, with clear explanations of the project's purpose, data use, and the right to withdraw.

**Transparency** — Feedback and grading decisions will be traceable, with tutors able to review rationale and transcripts, with explanations made available to students on request.

**Ethics Approval** — The project will obtain UNSW Human Research Ethics Committee (HREC) approval before deployment.

## 4.4 Initial Product Design

### 4.4.1 Personal AI tutor

#### Architecture Decisions

Figure 4.1 presents a high-level design in which a Model Context Protocol (MCP) server manages all interactions between the LLM and course resources. The MCP layer standardises context packaging (course materials, user/session state, system prompts) and mediates calls to external tools, enabling the tutor to answer student queries with grounded, auditable responses.

AWS services are a solid candidate to streamline security, observability, and scale. In this configuration, MCP endpoints are hosted behind an API gateway with serverless compute for tool adapters. Storage and search services back the retrieval flow, permissioned behind AWS Cognito.

A dedicated data-ingestion pipeline keeps the knowledge base updated. Course resources (lecture slides, specs, forum FAQs, exemplar solutions, policy documents) are uploaded, transformed and indexed into a vector store for later retrieval and search. Updates (e.g., new forum posts or amended specs) trigger re-indexing jobs so that retrieval remains consistent with the latest materials.

This architecture isolates capabilities behind MCP tools, simplifies policy enforcement (what the AI tutor is allowed to access and say), and supports easy future extension.

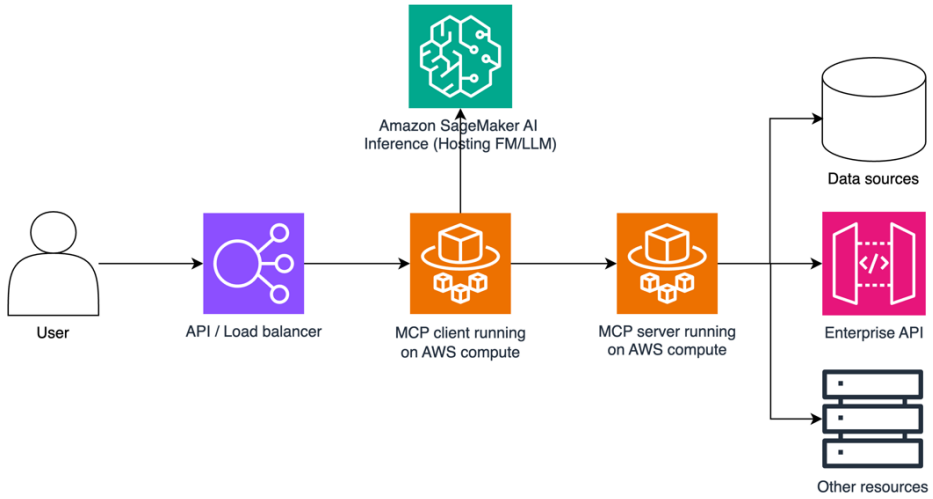


Figure 4.1: Potential architecture diagram for AI tutor using MCP

### User Stories and Acceptance Criteria

#### User Story 1: Context-Aware Question Answering

As a COMP9021 student, I want to ask the AI tutor questions about course content and get answers grounded in official materials, so that I receive accurate, relevant, and up-to-date explanations. *Acceptance Criteria:*

- Retrieves and references relevant sections of course materials.
- Responses are factually correct according to the latest course version.
- Cites sources or points to relevant lecture slides or problem sets.

#### User Story 2: Code Review and Feedback

As a student, I want the AI tutor to review my Python code and give feedback on logic, structure, and style, so that I can improve code quality before submission. *Acceptance Criteria:*

- Identifies syntax errors, logical issues, and poor coding practices.
- Feedback aligns with COMP9021 coding style guidelines.

- Offers at least one alternative solution or improvement.

**User Story 3: Practice Question Generation**

As a student, I want the AI tutor to generate practice questions tailored to my weak areas, so that I can focus my study more effectively. *Acceptance Criteria:*

- Uses quiz/assignment performance to identify weak areas.
- Matches difficulty to course assessment standards.
- Provides model solutions and explanations.

**User Story 4: Step-by-Step Problem Solving Guidance**

As a student, I want the AI tutor to guide me step-by-step through problem-solving, so that I can understand the reasoning process, not just the final answer. *Acceptance Criteria:*

- Breaks problems into logical, manageable steps.
- Explains the purpose of each step.
- Waits for student input before revealing the next step.

**User Story 5: Personalised Study Plan**

As a student, I want the AI tutor to create a personalised weekly study plan, so that I stay on track with course deadlines and revision. *Acceptance Criteria:*

- Includes lecture review, practice, and revision time.
- Syncs deadlines with COMP9021 assessment schedule.
- Updates automatically based on progress.

Table 4.1: Summary of User Stories for Personal AI Tutor

| # | **User Story** | **Acceptance Criteria (Summary)** |
|---|---|---|
| 1 | Personalised code feedback | Provides targeted, context-aware suggestions on student code; feedback is constructive and timely. |
| 2 | Hints without full solutions | Offers progressive, hint-based guidance; avoids giving complete solutions unless explicitly requested. |
| 3 | Course content integration | Uses COMP9021 materials in responses; ensures contextual relevance and accuracy. |
| 4 | Error detection and explanation | Identifies syntax and logical errors; provides clear explanations with examples. |
| 5 | Progress tracking | Monitors student performance; suggests topics for improvement and revision. |

### 4.4.2 Automated Oral Assessment System

#### Architecture Decisions

Diagram 4.2 represents a high-level overview of a potential implementation of the system. The end user inputs from a student are simply their assessment code submission and an oral examination. For teaching staff, they would need only to provide course teaching materials and sample solutions to the assessment.

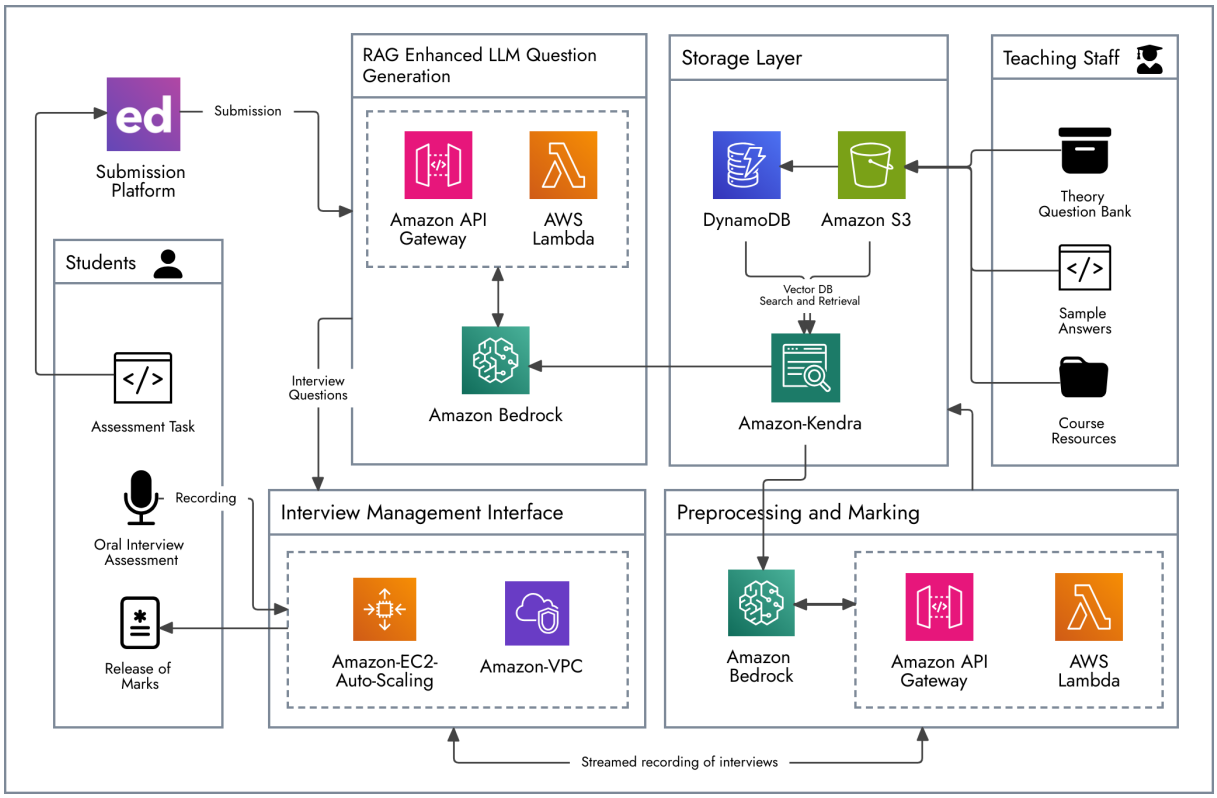


Figure 4.2: Potential architecture diagram for AI interviewer system

### User Stories and Acceptance Criteria

**User Story 1: Real-Time Oral Code Explanation** As a COMP9021 student, I want to be asked to explain my code during an automated interview, so that I can demonstrate my understanding of how it works. *Acceptance Criteria:*

- The system selects code segments from my submission.
- The system prompts me with targeted, open-ended questions.
- My spoken responses are recorded and transcribed accurately.

**User Story 2: Adaptive Questioning Based on Responses** As a COMP9021 student, I want the system to adapt its questions based on my answers, so that the interview reflects my actual understanding. *Acceptance Criteria:*

- Follow-up questions are contextually relevant to my last response.
- The difficulty of questions adjusts dynamically.
- The interview maintains an appropriate length and depth.

**User Story 3: Objective Scoring and Feedback** As a COMP9021 student, I want my oral responses to be scored consistently and objectively, so that I can trust the assessment process. *Acceptance Criteria:*

- The scoring system uses predefined rubrics aligned with course learning outcomes.
- Feedback highlights both strengths and areas for improvement.
- Scores are reviewed for consistency across multiple interviews.

**User Story 4: Bias-Aware Evaluation** As a COMP9021 student, I want the automated grading to minimise bias, so that my results are not affected by language background or accent. *Acceptance Criteria:*

- The system uses bias-mitigation techniques in evaluation.
- Performance is evaluated on content rather than speech fluency.
- A manual review option is available for disputed results.

**User Story 5: Integration with Course Assessment Systems** As a COMP9021 student, I want my interview results to integrate with the course's existing assessment platform, so that I have a single place to view all my grades. *Acceptance Criteria:*

- Scores and feedback are automatically uploaded to the LMS.
- Results are tagged with the relevant assessment identifier.
- The system maintains secure handling of all assessment data.

Table 4.2: Summary of User Stories for Automated Oral Assessment

| # | **User Story** | **Acceptance Criteria (Summary)** |
|---|---|---|
| 1 | Real-time oral code explanation | Selects code segments; asks open-ended questions; records and transcribes accurately. |
| 2 | Adaptive questioning | Adjusts difficulty and relevance of questions dynamically based on responses. |
| 3 | Objective scoring and feedback | Uses predefined rubrics; ensures consistency; provides constructive feedback. |
| 4 | Bias-aware evaluation | Minimises bias related to language or accent; allows manual review if needed. |
| 5 | LMS integration | Automatically uploads results to LMS; maintains secure handling of assessment data. |

## 4.5 Project Evaluation Plan

To measure the effectiveness and impact of the developed systems, a structured evaluation process will be conducted during the deployment phase. The primary method of data collection will be a questionnaire distributed to students enrolled in COMP9021 during the trial period. The questionnaire is designed to capture both quantitative and qualitative feedback on usability, perceived learning benefits, and the overall experience of interacting with the AI Tutor and Automated Oral Assessment systems.

The evaluation will focus on:

- Assessing improvements in learning outcomes and conceptual understanding.
- Measuring usability, accessibility, and satisfaction with the systems.
- Identifying potential challenges, limitations, and unintended consequences.
- Comparing student experiences between AI-assisted learning and traditional assessment methods.

**Proposed Questionnaire**

The questionnaire will combine Likert-scale questions, multiple-choice questions, and open-ended responses to provide both measurable and descriptive insights.

### 4.5.1 Part 1: AI Tutor Evaluation

1. How frequently did you use the AI Tutor during the course? (Never / Rarely / Sometimes / Often / Very often)
2. How helpful did you find the AI Tutor in understanding course concepts? (1 = Not helpful, 5 = Extremely helpful)
3. Did the AI Tutor improve your ability to solve problems independently? (Yes / No / Unsure)
4. Rate the clarity and accuracy of explanations provided by the AI Tutor. (1–5 scale)
5. What was the most useful feature of the AI Tutor? (Open-ended)
6. What improvements would you suggest for the AI Tutor? (Open-ended)

### 4.5.2 Part 2: Automated Oral Assessment Evaluation

1. How confident were you in explaining your code and reasoning during the oral assessment? (1–5 scale)
2. Did the oral assessment format help you better understand your own work? (Yes / No / Unsure)
3. How fair and accurate did you find the automated evaluation of your responses? (1–5 scale)
4. Did the oral assessment highlight areas for improvement that you were previously unaware of? (Yes / No)
5. What aspects of the oral assessment were most beneficial? (Open-ended)

6. What changes would make the oral assessment more effective? (Open-ended)

### 4.5.3 Part 3: Comparative and Overall Impact

1. Compared to traditional assessments, how effective were these tools in supporting your learning? (1–5 scale)
2. Which assessment method do you feel best demonstrated your understanding of the course material? (Multiple choice: AI Tutor tasks / Oral assessment / Traditional assignments / Mixed approach)
3. Do you believe these tools should be used in future course offerings? (Yes / No / Unsure)
4. Additional comments on your overall experience. (Open-ended)

Table 4.3: Mapping of questionnaire questions to project evaluation goals

| **Project Goal** | **Related Questionnaire Questions** |
|---|---|
| Assess improvements in learning outcomes | Part 1: Q2, Q3, Q4; Part 2: Q2, Q4; Part 3: Q1, Q2 |
| Measure usability and satisfaction | Part 1: Q1, Q2, Q4, Q5, Q6; Part 2: Q1, Q3, Q5, Q6; Part 3: Q3 |
| Identify potential limitations | Part 1: Q6; Part 2: Q6; Part 3: Q4 |
| Compare experiences between AI-assisted and traditional learning | Part 3: Q1, Q2, Q3 |

The collected data will be analysed using both statistical methods (for quantitative questions) and thematic coding (for qualitative responses) to generate a comprehensive evaluation of system performance and student reception.

# Chapter 5

# Results

# Chapter 6

# Conclusions

This initial report has established the background, motivation, and preliminary design for investigating the integration of AI tools into introductory programming education, using COMP9021 as a pilot context. The literature review identified both the opportunities—such as personalisation, improved feedback, and enhanced accessibility—and the challenges, including risks to academic integrity, over-reliance, and bias.

Preliminary planning has outlined two core applications for development: a course-specific AI tutor and an automated oral assessment framework. Technical options, such as the use of Retrieval-Augmented Generation (RAG) and Model Context Protocol (MCP) servers, have been explored to support these systems. A project methodology, ethical considerations, and an evaluation framework have also been defined.

The next stage of work will focus on prototype development, integration into the COMP9021 teaching environment, and structured evaluation with students. Findings from these stages will guide refinements and inform recommendations for responsible AI adoption in programming education.

# Bibliography

Ahmadi, A., Sharif, S. & Banad, Y. M. (2025), Mcp bridge: A lightweight, llm-agnostic restful proxy for model context protocol servers, Technical report, University of Oklahoma.
**URL:** *https://github.com/INQUIRELAB/mcp-bridge-api.*

Alpizar-Chacon (2025), Excited, skeptical, or worried? a multi-institutional study of student views on generative ai in computing education, *in* 'Proceedings of 25th International Conference on Computing Education Research, Koli Calling 2025', Association for Computing Machinery, Inc.

Andersen-Kiel, N. & Linos, P. (2024), Using chatgpt in undergraduate computer science and software engineering courses: A students' perspective, *in* 'Proceedings - Frontiers in Education Conference, FIE', Institute of Electrical and Electronics Engineers Inc.

Bakar, E. E. A., Halim, N. D. A., Hanid, M. F. A. & Inderawati, R. (2025), 'The challenges of teaching and learning programming in schools: Insights from a systematic literature review', *Karya Journal of Emerging Technologies in Human Services* **1**, 48–63.
**URL:** *https://karyailham.com.my/index.php/kjeths/article/view/203*

Bassner, P., Lenk-Ostendorf, B., Beinstingel, R., Wasner, T. & Krusche, S. (2026), 'Less stress, better scores, same learning: The dissociation of performance and learning in ai-supported programming education', *Computers and Education: Artificial Intelligence* **10**.

Beck, M. B. K. (2001), 'Manifesto for agile software development'.
**URL:** *https://agilemanifesto.org/*

Becker, D. (2023), 'Programming is hard – or at least it used to be: Educational opportunities and challenges of ai code generation'.

Bettayeb, A. M., Talib, M. A., Altayasinah, A. Z. S. & Dakalbab, F. (2024), 'Exploring the impact of chatgpt: conversational ai in education'.

Cambaz, D. & Zhang, X. (2024), Use of ai-driven code generation models in teaching and learning programming: a systematic literature review, *in* 'SIGCSE 2024 - Proceedings of the 55th ACM Technical Symposium on Computer Science Education', Vol. 1, Association for Computing Machinery, Inc, pp. 172–178.

Edwards-Fapohunda, M. O. & Adediji, M. A. (2024), Sustainable development of distance learning in continuing adult education: The impact of artificial intelligence, Technical report,

Global Banking School.
**URL:** *https://www.researchgate.net/publication/387945682*

Elkhatat, A. M., Elsaid, K. & Almeer, S. (2023), 'Evaluating the efficacy of ai content detection tools in differentiating between human and ai-generated text', *International Journal for Educational Integrity* **19**.

Elsayed, H. (2024), The impact of hallucinated information in large language models on student learning outcomes: A critical examination of misinformation risks in ai-assisted education, Technical report, South Valley University, Egypt.

Finnie-Ansley, J., Denny, P., Becker, B. A., Luxton-Reilly, A. & Prather, J. (2022), The robots are coming: Exploring the implications of openai codex on introductory programming, *in* 'ACM International Conference Proceeding Series', Association for Computing Machinery, pp. 10–19.

Gambo, I., Abegunde, F. J., Gambo, O., Ogundokun, R. O., Babatunde, A. N. & Lee, C. C. (2024), 'Grad-ai: An automated grading tool for code assessment and feedback in programming course', *Education and Information Technologies* .

Gray, S. L., Edsall, D. & Parapadakis, D. (2025), 'Ai-based digital cheating at university, and the case for new ethical pedagogies', *Journal of Academic Ethics* .

Hou, X., Zhao, Y., Wang, S. & Wang, H. (2025), 'Model context protocol (mcp): Landscape, security threats, and future research directions', *Huazhong University of Science and Technology* .
**URL:** *http://arxiv.org/abs/2503.23278*

Hu, C. (2023), 'Chatgpt sets record for fastest-growing user base'.
**URL:** *https://www.reuters.com/technology/chatgpt-sets-record-fastest-growing-user-base-analyst-note-2023-02-01/*

Huckins, G. (2025), 'Gpt-5 is here. now what? — mit technology review'.
**URL:** *https://www.technologyreview.com/2025/08/07/1121308/gpt-5-is-here-now-what/*

Humble, N., Boustedt, J., Holmgren, H., Milutinovic, G., Seipel, S. & Östberg, A. S. (2024), 'Cheaters or ai-enhanced learners: Consequences of chatgpt for programming education', *Electronic Journal of e-Learning* **22**, 16–29.

Jiao, H. (2025), Comparing human and ai rater effects using the many-facet rasch model, Technical report, University of Maryland.

Joseph, J. & Judy, M. V. (2024), Developing an intelligent integrated software agent for streamlined interview automation and enhanced candidate insights, *in* '2024 2nd DMIHER International Conference on Artificial Intelligence in Healthcare, Education and Industry, IDICAIEI 2024', Institute of Electrical and Electronics Engineers Inc.

Jukiewicz, M. (2024), 'The future of grading programming assignments in education: The role of chatgpt in automating the assessment and feedback process', *Thinking Skills and Creativity* **52**.

Kannam, S., Yang, Y., Dharm, A. & Lin, K. (2025), Code interviews: Design and evaluation of a more authentic assessment for introductory programming assignments, *in* 'SIGCSE TS 2025 - Proceedings of the 56th ACM Technical Symposium on Computer Science Education', Vol. 1, Association for Computing Machinery, Inc, pp. 554–560.

Lau, S. & Guo, P. (2023), From "ban it till we understand it" to "resistance is futile": How university programming instructors plan to adapt as more students use ai code generation and explanation tools such as chatgpt and github copilot, *in* 'ICER 2023 - Proceedings of the 2023 ACM Conference on International Computing Education Research V.1', Association for Computing Machinery, Inc, pp. 106–121.

Lee, H. (2024), 'The rise of chatgpt: Exploring its potential in medical education'.

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., tau Yih, W., Rocktäschel, T., Riedel, S. & Kiela, D. (2021), 'Retrieval-augmented generation for knowledge-intensive nlp tasks'.
**URL:** *http://arxiv.org/abs/2005.11401*

Liu, R., Zenke, C., Liu, C., Holmes, A., Thornton, P. & Malan, D. J. (2024), Teaching cs50 with ai: Leveraging generative artificial intelligence in computer science education, *in* 'SIGCSE 2024 - Proceedings of the 55th ACM Technical Symposium on Computer Science Education', Vol. 1, Association for Computing Machinery, Inc, pp. 750–756.

Lucia, A. D. & Qusef, A. (2010), 'Requirements engineering in agile software development', *Journal of Emerging Technologies in Web Intelligence* **2**, 212–220.

Min, S., Lyu, X., Holtzman, A., Artetxe, M., Lewis, M., Hajishirzi, H. & Zettlemoyer, L. (2022), 'Rethinking the role of demonstrations: What makes in-context learning work?', *Allen Institute for AI* .
**URL:** *http://arxiv.org/abs/2202.12837*

Parvathy, R., Thushara, M. G. & Kannimoola, J. M. (2025), 'Automated code assessment and feedback: A comprehensive model for improved programming education', *IEEE Access* .

Qureshi, B. (2023), Chatgpt in computer science curriculum assessment: An analysis of its successes and shortcomings, *in* 'ACM International Conference Proceeding Series', Association for Computing Machinery, pp. 7–13.

Raj, R. K., Becker, B. A., Goldweber, M. & Jalote, P. (2023), Perspectives on computer science curricula 2023 (cs2023), *in* 'CompEd 2023 - Proceedings of the ACM Conference on Global Computing Education', Vol. 2, Association for Computing Machinery, Inc, pp. 187–188.

Reckinger, S. J. & Reckinger, S. M. (2022), A study of the effects of oral proficiency exams in introductory programming courses on underrepresented groups, *in* 'SIGCSE 2022 - Proceedings of the 53rd ACM Technical Symposium on Computer Science Education', Vol. 1, Association for Computing Machinery, Inc, pp. 633–639.

Sheard, J., Simon, Butler, M., Falkner, K., Morgan, M. & Weerasinghe, A. (2017), Strategies for maintaining academic integrity in first-year computing courses, *in* 'Annual Conference on Innovation and Technology in Computer Science Education, ITiCSE', Vol. Part F128680, Association for Computing Machinery, pp. 244–249.

UNSW (2025), 'Human research ethics procedure'.
**URL:** *https://www.unsw.edu.au/content/dam/pdfs/governance/policy/2022-01-policies/humanresearchethicsprocedure.pdf*

Zviel-Girshin, R. (2024), 'The good and bad of ai tools in novice programming education', *Education Sciences* **14**.